

Bugs Sprint 2

Bug 01 — Email duplicado sin mensaje de error al usuario

Al intentar registrarse con un email ya existente, el backend respondía correctamente con un error 409, pero la interfaz no mostraba ningún feedback visual al usuario, dejándolo sin indicación de qué había salido mal.

Resuelto: se agregó manejo explícito del código 409 en el flujo de registro, mostrando un mensaje de error descriptivo en pantalla. El flujo de captura de errores es el mismo implementado para el Bug 01 del Sprint 1.

Bug 02 - Redirección post-login ignoraba el rol del usuario

Después de iniciar sesión exitosamente, la redirección no tenía en cuenta el rol del usuario autenticado, enviando a todos los usuarios a la misma ruta independientemente de si eran clientes o profesionales.

Resuelto: se modificó LoginScreen para leer tokens.role en el callback onSuccess y redirigir a la ruta correspondiente según el rol recibido en el token.

Bug 03 - ZodError al recargar la aplicación con token viejo sin campo id

Al recargar la aplicación con un token JWT persistido de una sesión anterior que no incluía el campo id, el parseo del token con Zod lanzaba un error que no se manejaba correctamente, interrumpiendo la inicialización del estado de autenticación.

Resuelto: se agregó id: z.number() al schema de validación del token. El try/catch existente en getInitialTokenState ya descartaba correctamente el token corrupto una vez que el schema rechazaba la estructura inválida.

Bug 04 - Conflictos de merge en archivos compartidos entre ramas

Los archivos Navigation.tsx, Login.ts y UserServices.ts eran modificados simultáneamente en distintas ramas, generando conflictos de merge que pisaban cambios entre compañeros y requerían resolución manual repetida.

Resuelto: se coordinaron los cambios sobre estos archivos entre los integrantes del equipo antes de mergear, estableciendo un orden de integración para evitar conflictos sobre las mismas secciones de código.

Bug 05 - Cambiar de tab en configuración descarta cambios sin advertencia

Si el usuario edita servicios en la pantalla de configuración y navega al tab "Horarios" sin guardar, el estado local de ServicesConfig se destruye silenciosamente y los cambios se pierden sin ningún aviso previo.

Pendiente: mostrar un diálogo de confirmación al intentar cambiar de tab con cambios sin guardar, o persistir el estado del formulario para que no se destruya al desmontar el componente.

Bug 06 - professional_id en el body de creación de servicios no se valida contra el usuario autenticado

El endpoint POST /services requiere que el frontend envíe el professional_id explícitamente en el cuerpo del request. Esto implica que el frontend debe conocer el ID del profesional antes de crear un servicio, y no existe ninguna validación en el backend que verifique que el professional_id recibido corresponde al usuario autenticado. Cualquier profesional podría crear servicios en nombre de otro simplemente modificando ese campo en el request.

Pendiente: eliminar professional_id del body del request y obtenerlo en el backend directamente desde el token JWT del usuario autenticado, garantizando que un profesional solo pueda crear servicios asociados a su propia cuenta.

Bug 07 - El calendario de reservas no bloquea fechas sin disponibilidad real

BookingCalendar deshabilita días pasados y días fuera del mes actual, pero no tiene forma de conocer qué días tienen slots disponibles antes de que el usuario seleccione una fecha. La disponibilidad real solo se consulta a través de useAvailability una vez que se elige un día, por lo que el usuario puede seleccionar cualquier fecha futura y recién entonces ver "Sin disponibilidad para esta fecha". Según los criterios de aceptación, el sistema debería sugerir la próxima fecha con turnos disponibles.

Pendiente: implementar una consulta de disponibilidad a nivel de mes o semana para poder deshabilitar visualmente los días sin slots antes de que el usuario los seleccione, y agregar sugerencia automática de la próxima fecha disponible.

Bug 08 - ServiceRestController expone la entidad de persistencia directamente como respuesta

ServiceRestController devuelve ServiceEntity directamente como response body, exponiendo campos internos como professional_id, slug e is_active con los nombres de columna de la base de datos, que no necesariamente coinciden con el contrato de la API documentada. Cualquier cambio en la entidad de persistencia rompe la API sin que se detecte hasta tiempo de ejecución.

Pendiente: introducir un DTO de respuesta (ServiceResponseDTO) que desacople la representación de la API de la entidad de persistencia, controlando explícitamente qué campos se exponen y con qué nombres.